

Excellent. **Part 3 (Questions 41–60)** covers **Next.js**, which is one of the most important topics for Senior Front-End interviews.

41. What is Next.js?

Answer

Next.js is a React framework that provides:

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Incremental Static Regeneration (ISR)
- API Routes
- File-based routing
- Image optimization
- Middleware
- Server Components
- Built-in SEO support

It improves performance, SEO, and developer experience.

42. Why use Next.js instead of React?

React	Next.js
UI Library	Full Framework
Client-side rendering by default	Multiple rendering strategies
Manual routing	File-based routing
Manual optimization	Built-in optimization
Manual SEO setup	Better SEO out of the box

Use React when building SPAs. Use Next.js when SEO, performance, and routing matter.

43. What is the App Router?

The App Router (introduced in Next.js 13) uses the `app/` directory instead of `pages/`.

Example:

```
app/
├─ layout.tsx
├─ page.tsx
├─ loading.tsx
├─ error.tsx
└─ blog/
    └─ [slug]/
        └─ page.tsx
```

Benefits:

- Nested layouts
- Server Components by default
- Better loading states
- Streaming
- Simpler data fetching

44. What are Server Components?

Server Components render on the server and send HTML to the browser.

Example:

TypeScript

```
export default async function Users() {
  const users = await fetch("/api/users").then(r => r.json());

  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

Benefits:

- Smaller JavaScript bundle
- Better SEO
- Faster initial load
- Can directly access databases or secure APIs on the server

Limitations:

- No `useState`
- No `useEffect`
- No browser APIs
- No event handlers

45. What are Client Components?

Client Components run in the browser and support interactivity.

```
TypeScript

"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Use Client Components when you need:

- State
- Effects
- Event handlers
- Browser APIs (`window`, `localStorage`, etc.)

46. Server Components vs Client Components

Server Component	Client Component
Runs on server	Runs in browser
Better SEO	Interactive
Smaller JS bundle	Larger bundle
Can fetch secure data	Cannot access server-only resources directly

Server Component

No hooks like `useState`

Client Component

Supports React hooks

Best practice: Use Server Components by default and Client Components only where interactivity is needed.

47. Explain SSR (Server-Side Rendering)

SSR generates HTML on every request.

Flow:

```
graph TD; Browser --> Server; Server --> FetchData[Fetch Data]; FetchData --> GenerateHTML[Generate HTML]; GenerateHTML --> SendHTML[Send HTML];
```

Pros:

- Excellent SEO
- Always fresh data

Cons:

- Slower than static pages
- Higher server load

Use for dashboards, user-specific pages, or frequently changing content.

48. What is SSG (Static Site Generation)?

Pages are generated at build time.

Flow:

```
graph TD; Build --> GenerateHTML[Generate HTML]; GenerateHTML --> Deploy;
```

↓
Serve Static Files

Pros:

- Extremely fast
- CDN-friendly
- Low server cost

Cons:

- Data isn't updated until the next build.

Use for blogs, documentation, and marketing pages.

49. What is ISR (Incremental Static Regeneration)?

ISR combines SSG with automatic regeneration.

Example:

TypeScript

```
export const revalidate = 60;
```

The page is regenerated in the background every 60 seconds after a request.

Benefits:

- Fast like SSG
- Keeps content relatively fresh
- Avoids rebuilding the whole application

50. SSR vs SSG vs ISR vs CSR

SSR	SSG	ISR	CSR
Every request	Build time	Background regeneration	Browser
Best SEO	Best SEO	Best SEO	Weak SEO
Fresh data	Static data	Semi-fresh	Fresh after API call
Slower	Fastest	Fast	Depends on network

Rule of thumb:

- **SSR**: Personalized or always-changing data.
 - **SSG**: Static content.
 - **ISR**: Mostly static content that updates periodically.
 - **CSR**: Highly interactive dashboards after login.
-

51. How does data fetching work in the App Router?

Server Components can fetch data directly:

TypeScript

```
const users = await fetch("https://api.example.com/users");
```

No need for `getServerSideProps` or `getStaticProps` in the App Router.

You control caching with options like:

TypeScript

```
fetch(url, {  
  cache: "no-store"  
});
```

or

TypeScript

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
});
```

52. Explain Next.js Fetch Caching

By default, Next.js caches fetch requests in Server Components.

Options:

Default

TypeScript

```
fetch(url);
```

Uses Next.js caching.

Disable cache

TypeScript

```
fetch(url, {  
  cache: "no-store"  
});
```

Always fetches fresh data.

Revalidate

TypeScript

```
fetch(url, {  
  next: {  
    revalidate: 300  
  }  
});
```

Refreshes cached data every 5 minutes.

53. What is Hydration?

Hydration is the process where React attaches JavaScript behavior to server-rendered HTML.

Flow:

```
Server HTML  
  ↓  
Browser loads HTML  
  ↓  
React loads JavaScript  
  ↓  
Hydration  
  ↓  
Interactive page
```

Without hydration, the page displays but buttons and forms won't work.

54. What is a Hydration Error?

A hydration error occurs when the HTML generated on the server doesn't match what React renders on the client.

Example:

TypeScript

```
<div>{Math.random()}</div>
```

The server and client generate different random values, causing a mismatch.

Common causes:

- `Date.now()`
- `Math.random()`
- Accessing `window` during rendering
- Different locale or timezone outputs

55. What is Middleware?

Middleware runs before a request reaches a page or API route.

Common uses:

- Authentication
- Redirects
- Rewrites
- Locale detection
- Security headers

Example:

TypeScript

```
import { NextResponse } from "next/server";

export function middleware(request) {
  if (!request.cookies.get("token")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
}
```

```
return NextResponse.next();  
}
```

56. How do you implement authentication in Next.js?

Typical flow:

```
Login  
↓  
Server verifies credentials  
↓  
Issue HttpOnly cookie  
↓  
Middleware checks cookie  
↓  
Allow or redirect
```

Best practices:

- Store JWT/session in **HttpOnly cookies**.
- Avoid storing sensitive tokens in `localStorage`.
- Protect routes with Middleware or server-side checks.

57. What are Layouts in the App Router?

Layouts allow UI to persist across routes.

Example:

```
app/  
  layout.tsx
```

A layout can contain:

- Navigation
- Sidebar
- Header
- Footer

Child pages are rendered inside the layout, avoiding unnecessary re-renders.

58. What are Loading and Error Components?

loading.tsx

Displays while a route is loading.

TypeScript

```
export default function Loading() {  
  return <p>Loading...</p>;  
}
```

error.tsx

Handles errors for a route segment.

TypeScript

```
"use client";  
  
export default function Error({ error }) {  
  return <p>{error.message}</p>;  
}
```

This provides route-level loading and error boundaries.

59. How do you optimize performance in Next.js?

Common techniques:

- Use Server Components where possible.
- Minimize Client Components.
- Use dynamic imports:

TypeScript

```
const Chart = dynamic(() => import("./Chart"));
```

- Optimize images with `next/image`.
 - Use font optimization.
 - Cache API requests appropriately.
 - Use code splitting.
 - Virtualize long lists.
 - Analyze bundles with `@next/bundle-analyzer`.
-

60. Explain Image Optimization in Next.js

Instead of a normal `<img>` tag:

TypeScript

```
import Image from "next/image";

<Image
  src="/hero.jpg"
  alt="Hero"
  width={800}
  height={400}
/>
```

Benefits:

- Lazy loading
- Automatic resizing
- Responsive images
- Modern formats (WebP/AVIF when supported)
- Reduced layout shift
- Better Core Web Vitals

Senior Interview Follow-up Questions

Why are Server Components the default in the App Router?

Because they reduce the JavaScript sent to the client, improve performance and SEO, and can safely access server-side resources.

When would you choose `cache: "no-store"`?

For data that must always be fresh, such as stock prices, user-specific dashboards, or real-time inventory.

Why use Middleware instead of checking authentication in every page?

Middleware centralizes authentication logic, avoids duplication, and blocks unauthorized requests before rendering.

What is the biggest mistake developers make with Client Components?

Marking entire pages with `"use client"` when only a small interactive section requires it. This increases bundle size and reduces the benefits of Server Components.

These Next.js concepts are frequently discussed in senior interviews because they test your understanding of rendering strategies, performance, caching, and application architecture—not just API knowledge. The next section will cover **TypeScript (Generics, `keyof`, utility types, interfaces vs. types, type guards, and advanced typing)**.